

Problem Set A — Python Foundations (Weeks 1–2)

Programming and Numerical Methods for Economics (ECNM)

The University of Edinburgh · School of Economics

Instructions. This problem set covers material from Weeks 1 and 2: Python basics, NumPy arrays, loops, conditionals, functions, data types, tuples, and dictionaries. Answer every question in the code cells provided. Where a question asks you to *explain*, write your answer as a comment or a Markdown cell.

This version includes solutions.

```
In [ ]: import numpy as np
```

Question — Lists and basic operations

A country's quarterly real GDP growth rates (annualised, %) over two years are:

Q	Q	Q	Q	Q	Q	Q	Q
1	2	3	4	1	2	3	4
2019	2019	2019	2019	2020	2020	2020	2020

(a) Store these values in a Python list called `gdp_growth`.

(b) Using only list indexing (no NumPy), extract the growth rates for the **second year** (Q1–Q4 2020) into a new list called `year2`.

(c) Write a `for` loop that counts how many quarters had **negative** growth. Store the count in a variable called `n_neg`.

(d) Without using any built-in `sum()` or NumPy, write a loop that computes the **average** growth rate across all eight quarters.

```
In [ ]: # --- Solution ---

# (a)
gdp_growth = [2.1, 1.8, -0.3, -1.2, 0.5, 1.4, 2.0, 2.5]

# (b)
year2 = gdp_growth[4:]
print("Year 2 growth rates:", year2)

# (c)
n_neg = 0
```

```

for g in gdp_growth:
    if g < 0:
        n_neg += 1
print("Quarters with negative growth:", n_neg)

# (d)
total = 0
for g in gdp_growth:
    total += g
avg = total / len(gdp_growth)
print("Average growth rate:", avg)

```

Question — NumPy arrays and vectorised operations

Suppose two sectors of an economy produce output vectors (in billions of £) over five years:

- **Manufacturing:** [120, 115, 118, 125, 130]
- **Services:** [200, 210, 205, 220, 240]

- Create two NumPy arrays, `mfg` and `svc`, from these lists.
- Compute `total_output`, the element-wise sum of the two sectors for each year.
- Compute `mfg_share`, the manufacturing share of total output for each year (as a proportion percentage).
- Using NumPy, compute the **year-on-year growth rate** of total output for years $t = 1, 2, 3, 4$. Store `growth_total`. (Hint: the growth rate from year t to $t+1$ is $(y_{t+1} - y_t)/y_t$.)
- What is the average manufacturing share? What is the maximum growth rate of total output? Use NumPy functions.

```

In [ ]: # --- Solution ---

# (a)
mfg = np.array([120, 115, 118, 125, 130])
svc = np.array([200, 210, 205, 220, 240])

# (b)
total_output = mfg + svc
print("Total output:", total_output)

# (c)
mfg_share = mfg / total_output
print("Manufacturing share:", mfg_share)

# (d)
growth_total = (total_output[1:] - total_output[:-1]) / total_output[:-1]
print("Year-on-year growth of total output:", growth_total)

# (e)

```

```
print("Average manufacturing share:", np.mean(mfg_share))
print("Maximum growth rate of total output:", np.max(growth_total))
```

Question — Functions and economic application

(a) Write a function `cobb_douglas(K, L, A=1, alpha=0.33)` that returns output $Y = A K^{\alpha} L^{1-\alpha}$. Use default parameter values as shown.

(b) Evaluate the function at $K=100$, $L=200$ with default A and α . Then evaluate again with $A=1.5$.

(c) Write a function `returns_to_scale(K, L, A=1, alpha=0.33, scale=2)` that:

- Computes $Y_{\text{default}} = \text{cobb_douglas}(K, L, A, \alpha)$.
- Computes $Y_{\text{tfp}} = \text{cobb_douglas}(\text{scale} \cdot K, \text{scale} \cdot L, A, \alpha)$.
- Returns the ratio $Y_{\text{tfp}} / Y_{\text{default}}$.

What does your function return for `scale=2`? Explain why, given the Cobb–Douglas exponents.

```
In [ ]: # --- Solution ---

# (a)
def cobb_douglas(K, L, A=1, alpha=0.33):
    return A * K**alpha * L**(1 - alpha)

# (b)
Y_default = cobb_douglas(100, 200)
print("Y (A=1):", round(Y_default, 2))

Y_tfp = cobb_douglas(100, 200, A=1.5)
print("Y (A=1.5):", round(Y_tfp, 2))

# (c)
def returns_to_scale(K, L, A=1, alpha=0.33, scale=2):
    Y1 = cobb_douglas(K, L, A, alpha)
    Y2 = cobb_douglas(scale * K, scale * L, A, alpha)
    return Y2 / Y1

ratio = returns_to_scale(100, 200)
print("Y2 / Y1 when inputs double:", round(ratio, 4))
# The ratio is 2.0 because alpha + (1-alpha) = 1 → constant returns to scale
# Doubling both inputs exactly doubles output.
```

Question — Conditionals, loops, and dictionaries

An economist classifies countries by GDP per capita (in thousands of USD):

Country	GDP per capita
Argentina	13.7
United Kingdom	46.5
Nigeria	2.1
South Korea	34.8
Ethiopia	1.0
Germany	51.2

(a) Store this data in a dictionary called `gdp_pc` where keys are country names and values are per capita.

(b) Write a loop that iterates over the dictionary and classifies each country as:

- 'Low income' if GDP per capita < 4
- 'Middle income' if $4 \leq$ GDP per capita < 20
- 'High income' if GDP per capita ≥ 20

Store the results in a new dictionary called `classification` with the same keys.

(c) Using your `classification` dictionary, count how many countries fall into each income group. Print the counts.

```
In [ ]: # --- Solution ---

# (a)
gdp_pc = {
    'Argentina': 13.7,
    'United Kingdom': 46.5,
    'Nigeria': 2.1,
    'South Korea': 34.8,
    'Ethiopia': 1.0,
    'Germany': 51.2
}

# (b)
classification = {}
for country, gdp in gdp_pc.items():
    if gdp < 4:
        classification[country] = 'Low income'
    elif gdp < 20:
        classification[country] = 'Middle income'
    else:
        classification[country] = 'High income'

print("Classifications:")
for country, cat in classification.items():
    print(f" {country}: {cat}")

# (c)
counts = {'Low income': 0, 'Middle income': 0, 'High income': 0}
for cat in classification.values():
    counts[cat] += 1
```

```
print("\nCounts:")
for group, n in counts.items():
    print(f" {group}: {n}")
```

Question — Numerical integration and while loop

The **consumer surplus** for a linear demand curve $P(Q)=a - bQ$ above a market price P^* is the area of the triangle between the demand curve and the price line, from $Q=0$ to the equilibrium quantity $Q^* = (a - P^*)/b$.

The exact consumer surplus is: $CS = \int_0^{Q^*} [P(Q) - P^*] dQ = \frac{(a - P^*)^2}{2b}$

Set $a = 10$, $b = 0.5$, $P^* = 3$.

(a) Compute the exact consumer surplus using the formula.

(b) Write a function `cs_riemann(a, b, p_star, n)` that approximates the consumer surplus using a **left Riemann sum** with `n` rectangles. (Hint: divide $[0, Q^*]$ into `n` equal sub-intervals; sub-interval evaluate $P(Q_i) - P^*$ at the left endpoint and multiply by the width ΔQ .)

(c) Using a `while` loop, find the **smallest** value of `n` such that the Riemann approximation is within 0.01 of the exact answer. Print `n` and the approximation.

```
In [ ]: # --- Solution ---

a, b, p_star = 10, 0.5, 3

# (a) Exact consumer surplus
Q_star = (a - p_star) / b
CS_exact = (a - p_star)**2 / (2 * b)
print(f"Exact CS = {CS_exact}")
print(f"Equilibrium quantity Q* = {Q_star}")

# (b) Riemann sum approximation
def cs_riemann(a, b, p_star, n):
    Q_star = (a - p_star) / b
    dQ = Q_star / n
    cs = 0
    for i in range(n):
        Q_i = i * dQ
        P_i = a - b * Q_i
        cs += (P_i - p_star) * dQ
    return cs

print(f"Riemann (n=10): {cs_riemann(a, b, p_star, 10):.4f}")
print(f"Riemann (n=100): {cs_riemann(a, b, p_star, 100):.4f}")

# (c) While loop to find smallest n within tolerance
n = 1
while abs(cs_riemann(a, b, p_star, n) - CS_exact) > 0.01:
```

```

n += 1

print(f"\nSmallest n within 0.01 of exact: n = {n}")
print(f"Riemann approximation: {cs_riemann(a, b, p_star, n):.6f}")
print(f"Exact value: {CS_exact:.6f}")

```

Question — Tuples, unpacking, and a mini proje

A bond pays fixed annual coupons. Write a function `bond_value(face, coupon_rate, r, T)` that:

- Computes the **present value** of all coupon payments: $SPV_{\{\text{coupons}\}} = \sum_{t=1}^T \frac{C}{(1+r)^t}$, where $C = \text{face} \times \text{coupon_rate}$.
- Computes the **present value** of the face value repaid at maturity: $SPV_{\{\text{face}\}} = \frac{\text{face}}{(1+r)^T}$.
- Returns a **tuple** `(pv_coupons, pv_face, total_pv)` where `total_pv = pv_coupons + pv_face`.

(a) Write the function.

(b) Call it with `face=1000`, `coupon_rate=0.05`, `r=0.03`, `T=10`. Unpack the result into variables and print each.

(c) What happens to the total bond value if the market interest rate `r` rises to `0.07`? Is the bond trading at a premium or a discount? Explain in one sentence.

```

In [ ]: # --- Solution ---

# (a)
def bond_value(face, coupon_rate, r, T):
    C = face * coupon_rate
    pv_coupons = sum(C / (1 + r)**t for t in range(1, T + 1))
    pv_face = face / (1 + r)**T
    total_pv = pv_coupons + pv_face
    return (pv_coupons, pv_face, total_pv)

# (b)
pv_c, pv_f, total = bond_value(1000, 0.05, 0.03, 10)
print(f"PV of coupons: £{pv_c:.2f}")
print(f"PV of face:    £{pv_f:.2f}")
print(f"Total PV:      £{total:.2f}")

# (c)
pv_c2, pv_f2, total2 = bond_value(1000, 0.05, 0.07, 10)
print(f"\nTotal PV at r=0.07: £{total2:.2f}")
# The bond is now trading at a discount (PV < face value = £1000)
# because the market rate exceeds the coupon rate – investors demand
# a price below par to accept the lower coupon.

```

End of Problem Set A.